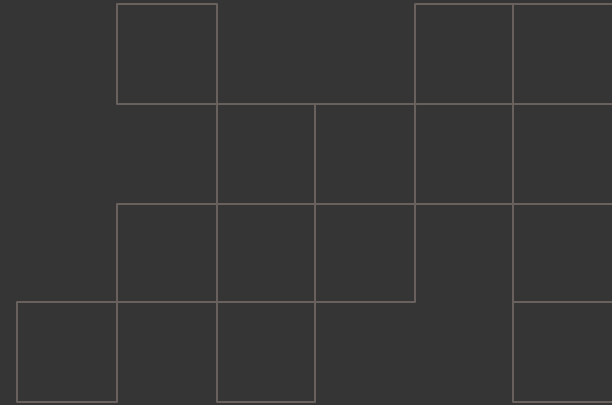


Proyecto Final: Auto- etiquetación de imágenes

Autor: Jorge De Diego Jiménez

Ciclo: CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)

Curso: 25-26



Índice

1.
Introducción
2.
Análisis de Mercado y Propuesta de Valor
3.
Diseño UX/UI y Flujo de Usuario
4.
Stack Tecnológico
5.
Arquitectura del Sistema
6.
Presupuesto y Monetización
7.
Conclusiones y Líneas de Futuro

Introducción

Scryframe es una aplicación web de catalogación de imágenes pensada para uso local, orientada a artistas, ilustradores y coleccionistas que necesitan organizar grandes colecciones personales.

Problema: Mantener manualmente miles de imágenes etiquetadas es inviable.

Solución: Delegar el etiquetado inicial a un modelo de IA (DeepDanbooru) que propone tags descriptivos. El usuario confirma o ajusta antes de guardar.

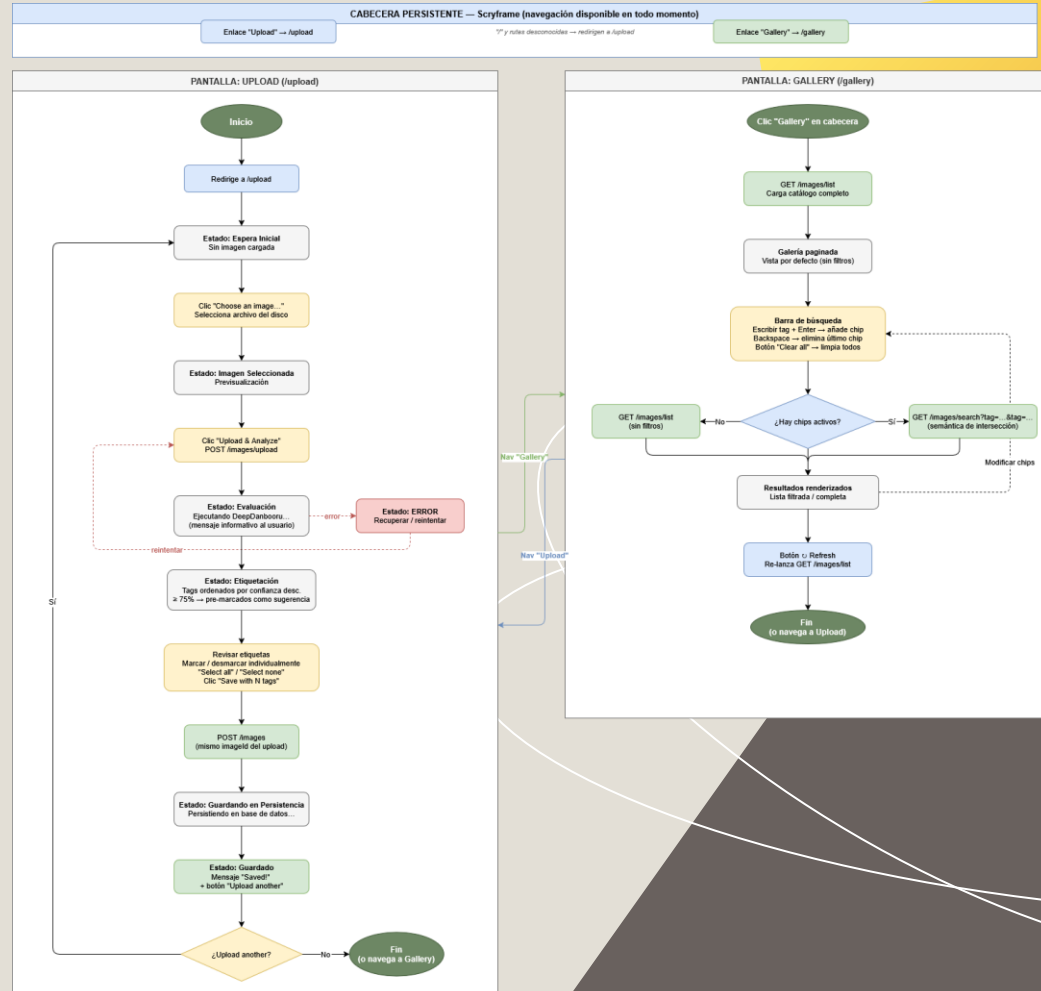
Flujo principal: Subida de imagen → predicción automática de etiquetas → confirmación → galería paginada y filtrable.



Análisis de Mercado

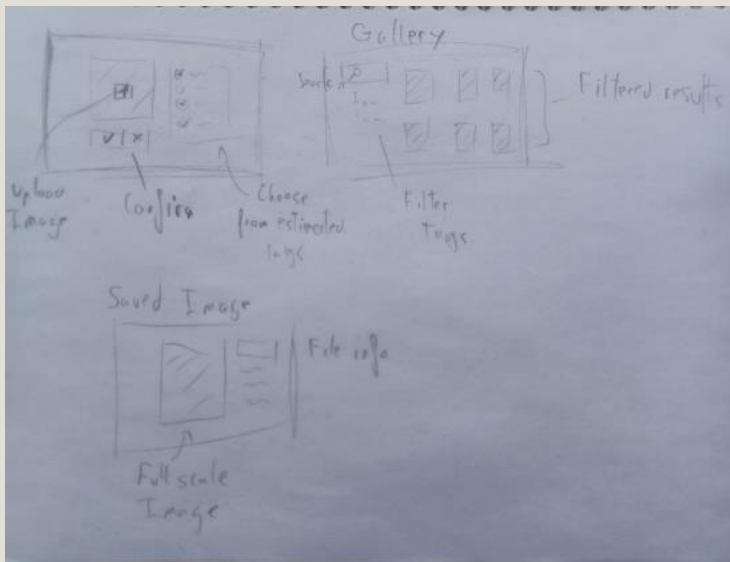
Scryframe ocupa un hueco entre la complejidad de Hydrus y el coste de Eagle:

Competidor	Limitación cubierta por Scryframe
Hydrus Network	Sin etiquetado IA; interfaz muy técnica y curva de aprendizaje pronunciada.
Eagle	Cerrado y de pago (~30 €); sin etiquetado automático real.
Scryframe	IA integrada, local-first, interfaz sencilla e instalación gratuita.

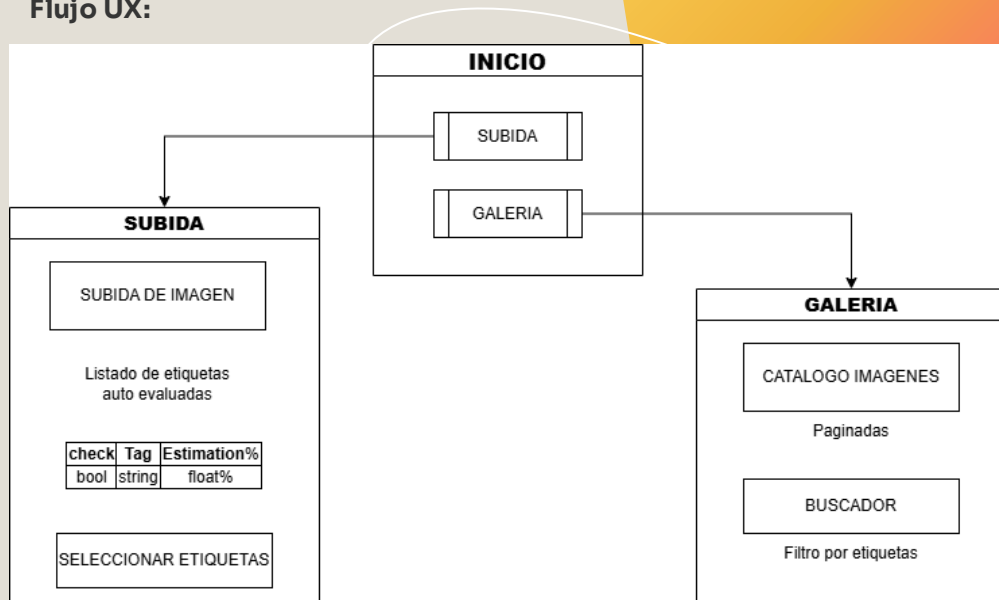


Diseño UX/UI

Fidelidad Baja:

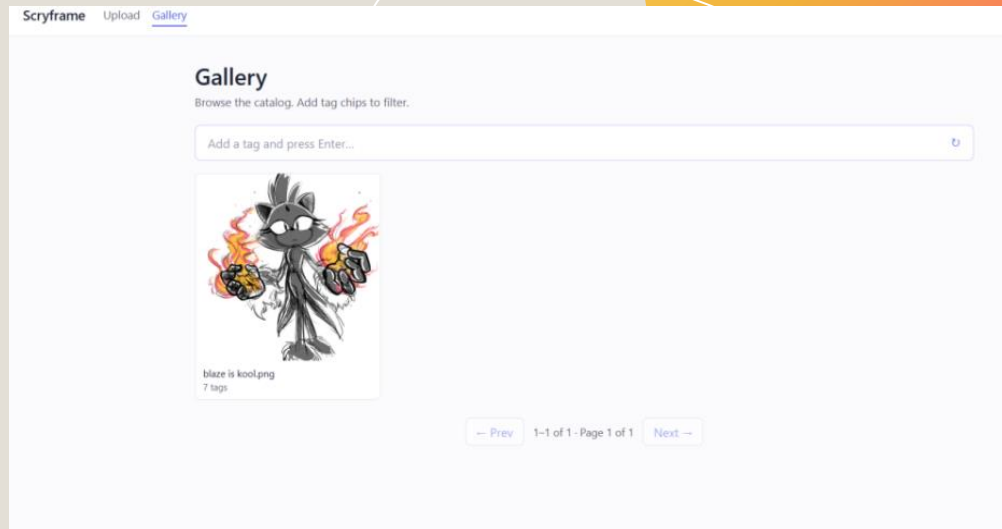
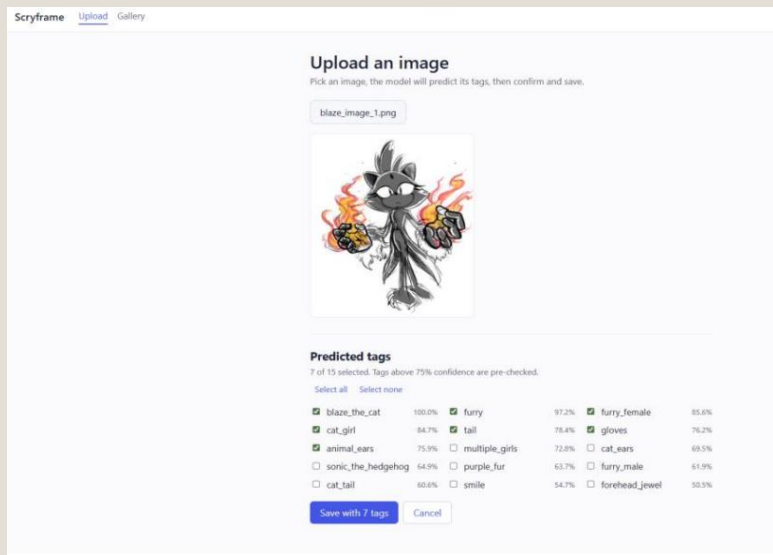


Flujo UX:



Diseño UX/UI

Fidelidad Alta:



Stack Tecnológico

El proyecto usa un stack moderno elegido para integrar tres tecnologías heterogéneas:

Backend: Java 25 · Spring Boot 4.0.6 · JPA/Hibernate · H2 · Lombok · Maven

Frontend: TypeScript 5.9 · Angular 20.3 · RxJS 7.8 · SCSS · Angular CLI

Modelo IA: DeepDanbooru · Python 3.11

Herramientas: IntelliJ IDEA · WebStorm · Git · Node.js 22+ · npm 10+



Arquitectura del Sistema

Scryframe sigue una arquitectura desacoplada en tres capas comunicadas exclusivamente via HTTP/REST:

Frontend: SPA Angular 20.3 con componentes standalone. HttpClient + RxJS 7.8 + Signals. Dos pantallas: /upload y /gallery.

Backend: API REST Spring Boot 4 con arquitectura hexagonal (DDD). Capas: domain, application, infrastructure.

DeepDanbooru (IA): Modelo ResNet ejecutado como subproceso cmd.exe. Salida parseada con regex para obtener tags con confianza.



Presupuesto y Monetización

Modelo de negocio Freemium con opción Premium a 3 €/mes (catálogo ilimitado + exportación de tags).

Ingresos base: 200 usuarios premium x 3 €/mes = 7.200 €/año (escenario base).

Inversión inicial: 4.800 € (amortización contable a 3 años del equipo de desarrollo).

Break-even: ~8 meses · ROI primer año: ~50%.

Conservador (100 usuarios): -25% ROI | Optimista (400 usuarios): +200% ROI, break-even 4 meses.

Conclusiones y Futuro

Se cumple el PMV: subida de imagen, predicción automática, confirmación, persistencia en disco y galería paginada filtrable por tags.

Logros: Integración exitosa de Spring Boot (DDD) + Angular + DeepDanbooru via REST. Arquitectura modular y extensible.

Dificultades: Implementación de DeepDanbooru (desactualizado), parseo robusto de su salida, y gestión del imageld entre los endpoints upload/save.

Líneas de futuro: Paginación server-side, thumbnails, edición post-guardado, vista de detalle, microservicio Python REST, soporte multi-usuario.

